

Openaerostruct 教程

安装教程：此安装仍然基于 ubuntu 环境下
建议大家创建虚拟环境中安装，避免包冲突。

例如：

创建虚拟环境：python3 -m venv oas_env

这将在当前目录下创建一个名为 oas_env 的虚拟环境。

使用 source 激活虚拟环境：source oas_env/bin/activate

然后在激活的虚拟环境中进行安装

1. 最简单的方法是直接从 PyPI 中直接安装：

```
pip install openaerostruct
```

2. 当然也提供了另一种更轻松的访问示例，直接克隆 OpenAeroStruct 存储库来安装
OpenAeroStruct

```
git clone https://github.com/mdolab/OpenAeroStruct.git
```

然后从 OpenAeroStruct 文件夹中，pip install 此软件包

```
cd openaerostruct
```

```
pip install -e .
```

这两种方法都会自动安装依赖项：numpy, scripy,matplotlib,和 OpenMDAO.

使用分析教程：

详细的参考见：

https://mdolab-openaerostruct.readthedocs-hosted.com/en/latest/struct_example.html

OpenAeroStruct 是一个开源的、用于飞机结构气动弹性优化的软件包。它紧密地结合了空气动力学和结构力学分析，能够在优化机翼形状的同时，考虑结构变形和重量带来的影响。它填补了快速概念设计与高保真仿真之间的空白。它通过将气动和结构分析紧密耦合，使工程师能够在设计初期就找到气动性能与结构重量之间最佳平衡的方案，是进行飞机机翼多学科设计优化的优秀平台。

下面给出一个简单的结构求解示例程序：

3.1 导入模块

```
import numpy as np

from openaerostruct.meshing.mesh_generator import generate_mesh
from openaerostruct.structures.struct_groups import SpatialBeamAlone

import openmdao.api as om
```

3.2 几何定义和网格生成

```
mesh_dict = {"num_y": 7, "wing_type": "CRM", "symmetry": True, "num_twist_cp": 5}

mesh, twist_cp = generate_mesh(mesh_dict)
```

- 创建一个 CRM (Common Research Model) 标准研究机翼的半模 (symmetry: True)
- 沿展向划分 7 个节点 (num_y: 7)
- 生成参数化网格，用于后续的气动和结构分析

当然，他也支持自定义网格，可详细参考教程：

3.3 结构属性的定义

```
surf_dict = {}  
# Wing definition  
"name": "wing", # name of the surface  
"symmetry": True, # if true, model one half of wing  
# reflected across the plane y = 0  
"fem_model_type": "tube", # 管状的结构  
"mesh": mesh,  
# Structural values are based on aluminum 7075  
"E": 70.0e9, # [Pa] Young's modulus of the spar, 主梁杨氏模量, 表达材料的刚度  
"G": 30.0e9, # [Pa] shear modulus of the spar, 剪切模量, 表达材料的剪切模量  
"yield": 500.0e6 / 2.5, # [Pa] yield stress divided by 2.5 for limiting case, 安全因子/屈服极限  
"rho": 3.0e3, # [kg/m^3] material density, 材料密度  
"fem_origin": 0.35, # normalized chordwise location of the spar, 梁的模型位置 (弦向), 0表示前缘, 1位后缘, 建议0.3到0.4  
"t_over_c_cp": np.array([0.15]), # maximum airfoil thickness 最大翼型厚度比, 只设置一个点, 指定机翼各段的翼型厚度比  
"thickness_cp": np.ones(3) * 0.1, # 梁厚度控制点  
"wing_weight_ratio": 2.0, # 翼量修正系数, 用于估算安装件, 配线以及油箱等附加重量  
"struct_weight_relief": False, # True to add the weight of the structure to the loads on the structure, 是否考虑结构自重影响, 若考虑, 则将结构重力反作用计入载荷  
"distributed_fuel_weight": False, # 是否模拟燃油重量分布  
"exact_failure_constraint": False, # 是否使用精确应力计算
```

这些参数定义了机翼的结构和材料属性

3.4 载荷的定义

```
indep_var_comp = om.IndepVarComp()  
indep_var_comp.add_output("loads", val=np.ones((ny, 6)) * 2e5, units="N") #表示每个截面的上的6个自由度载荷, Fx, Fy, Fz, Mx, My, Mz  
indep_var_comp.add_output("load_factor", val=1.0) #表示过载系数, 表示几倍载荷
```

在每个节点施加 200000N 的六自由度载荷 (Fx, Fy, Fz, Mx, My, Mz)
载荷因子为 1.0——正常飞行状态

3.5 结构组创建与系统集成

```
struct_group = SpatialBeamAlone(surface=surf_dict)
```

这是一个复杂的多组件系统, 包含了以下子系统: **geometry**: 几何变换和网格处理
local_stiff: 局部刚度矩阵计算, **fem**: 有限元组装和求解, **stress**: 应力计算和失效分析
constraints: 约束条件计算, **mass**: 质量计算

3.6 变量连接以及提升机制

```
struct_group.add_subsystem("indep_vars", indep_var_comp, promotes=["*"])
```

这句话把 indep_vars 中的所有变量提升到 struct_group 的命名空间, 并且实现自动连接

3.7 模型层级结构

```
prob.model.add_subsystem(surf_dict["name"], struct_group)
```

结构命名为 wing.

3.8 优化器配置详解

```
prob.driver = om.ScipyOptimizeDriver()  
prob.driver.options["disp"] = True  
prob.driver.options["tol"] = 1e-9
```

3.9 设计变量详细分析以及问题求解

```

prob.model.add_design_var("wing.thickness_cp", lower=0.01, upper=0.5, ref=1e-1) #设计变量
prob.model.add_constraint("wing.failure", upper=0.0) #设计约束
prob.model.add_constraint("wing.thickness_intersects", upper=0.0)

# Add design variables, constraint, and objective on the problem
prob.model.add_objective("wing.structural_mass", scaler=1e-5)#缩放因子, 设置优化目标

# Set up the problem
prob.setup()

prob.run_driver()

```

设定设计参数,约束条件以及目标函数,为所有的变量分配内存空间以及设置解析导数,验证所有的变量连接的正确性,构建完整的计算,并求解。

3.10 可视化

Openaerostruct 提供了两种机翼结构模型,管状翼梁和翼盒,即 tube 和 wingbox 两种模型:我们可以在程序中加载和使用.db 文件,(即 OpenMDAO 的 SQLite 录制器记录的数据库文件)

OpenAeroStruct 会将求解过程和变量结果记录到一个 SQLite 数据库中(后缀 .db),常用于:

存储每一次迭代的设计变量 / 目标函数 / 约束值

支持后处理、绘图分析、调试查看中间值

如何让程序输出.db 文件呢?

在 OpenMDAO 中,你需要添加一个 SqliteRecorder 到 driver 上,如下所示:

```

import openmdao.api as om

recorder = om.SqliteRecorder("struct.db") # 指定保存路径
prob.driver.add_recorder(recorder)

# 可选项: 记录导数/包含哪些变量
prob.driver.recording_options["record_derivatives"] = True
prob.driver.recording_options["includes"] = ["*"]

```

然后,使用系统自带的内置绘制函数,针对于不同的结构,可以进行结构的绘制以及优化过程的记录,实现可视化,程序位于安装的 `openaerostruct/utlis/plot_wing.py` 或者 `plot_wingbox.py` 程序。

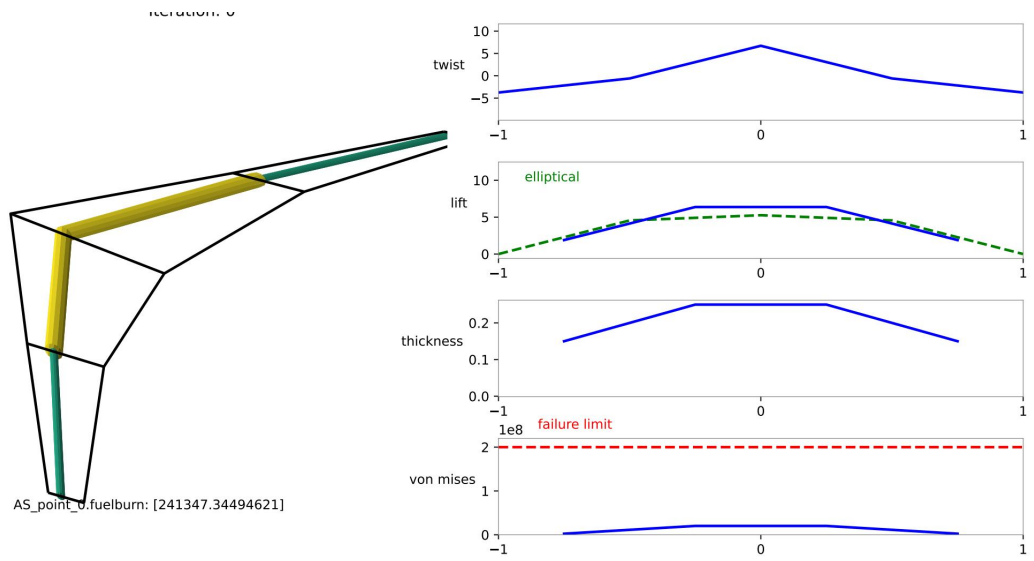
在 `openaerostruct/examples` 文件夹中提供了很多个示例程序,包含耦合优化以及创建 db 文件的过程,大家可以参考

绘制过程:(确保已经生成 db 文件)

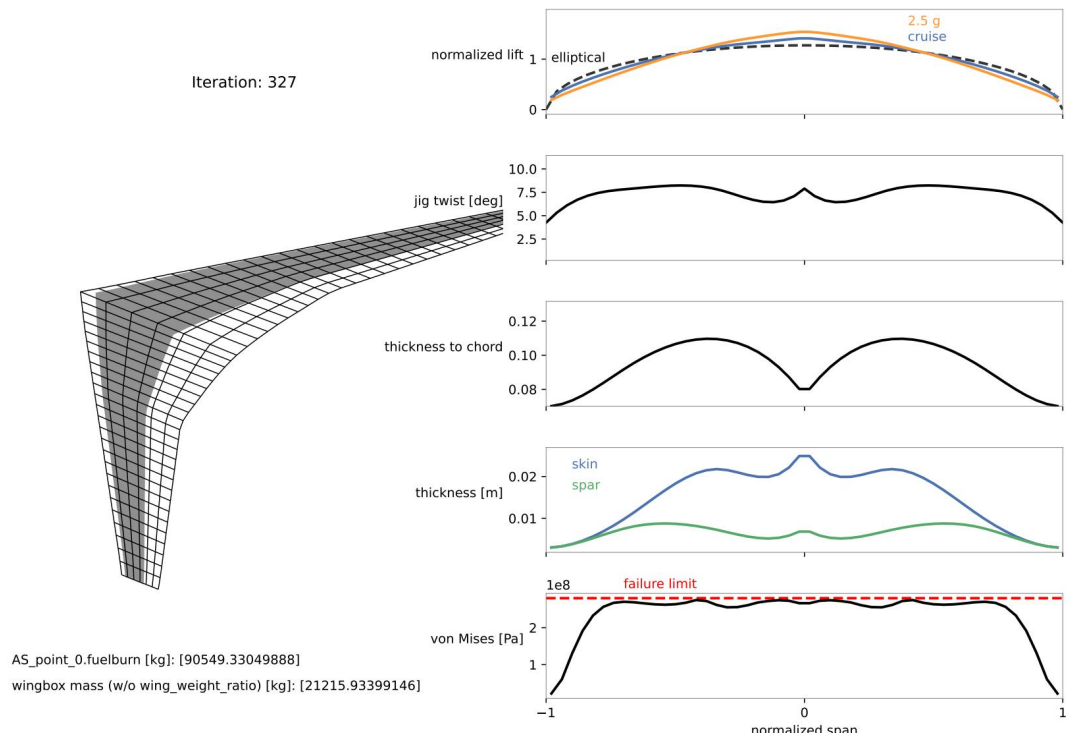
例: `python3 plot_wing.py struct.db`

我们生成优化示例:如下图

Tube:



Wingbox



注：所有运行均基于 ubuntu 系统环境下。

1. 可以直接安装 ubuntu 系统进行程序的运行，或者使用虚拟机